AWS Cross-Account Deployment with Terraform

KangZheng Li · Subscribe
4 min read · Sep 1, 2023

24 1

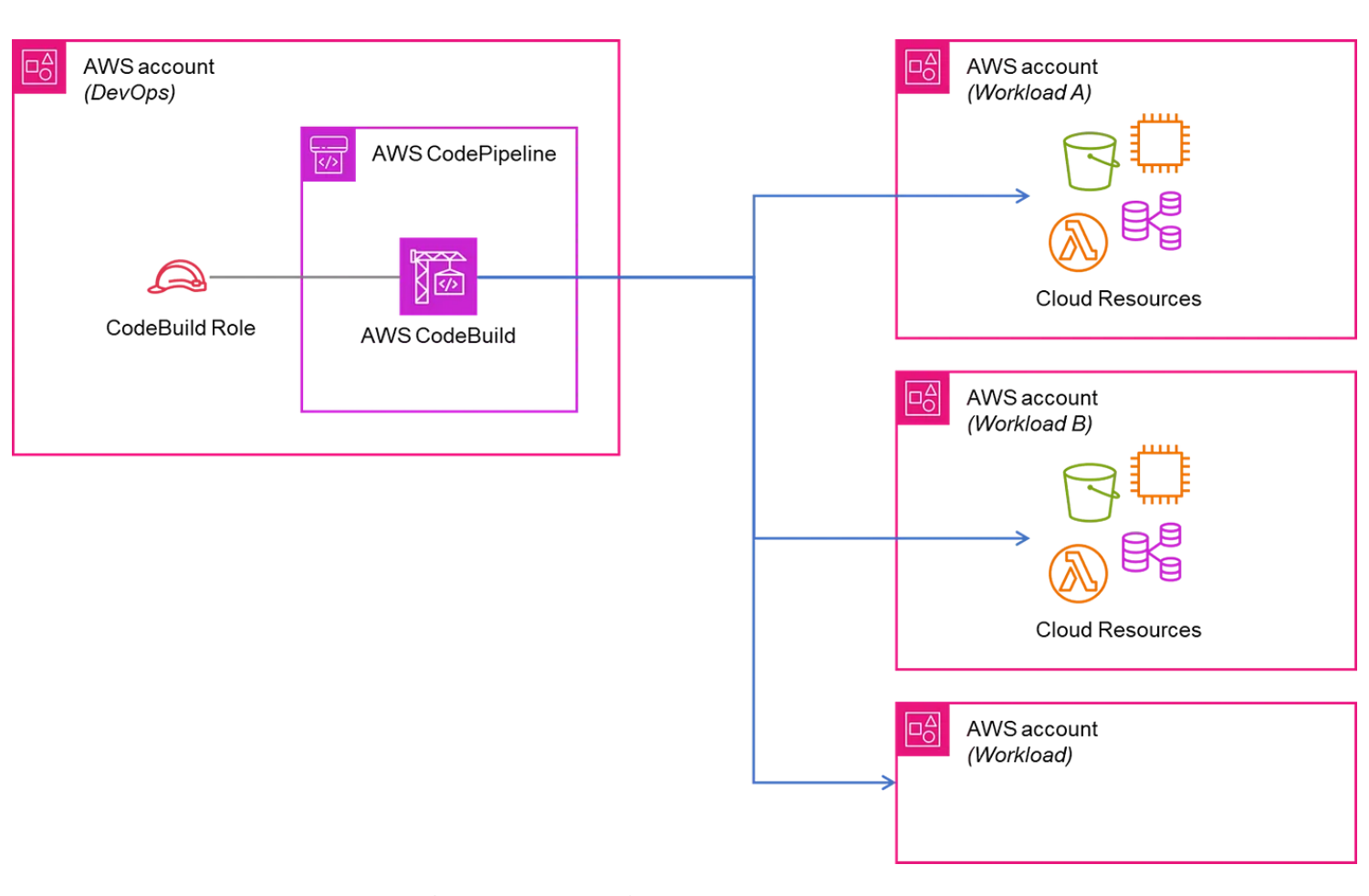
In this article, we will explore using Terraform to do cross-account deployment in AWS.

What is Cross-Account deployment?

Cross-account deployment is an approach to deploying AWS resources in one Account from another isolated account.

It is common for organisations to own more than one AWS account. However, deploying resources into these accounts individually can be difficult and unscalable. A solution for this would be through centralisation and automation.

Infrastructure-as-Code (IaC) tools such as Terraform are usually executed from a centralised DevOps account, orchestrating the CI/CD processes and deploying resources into other AWS accounts. Through CI/CD processes, we can incorporate best practices such as Policy-as-Code like Checkov.



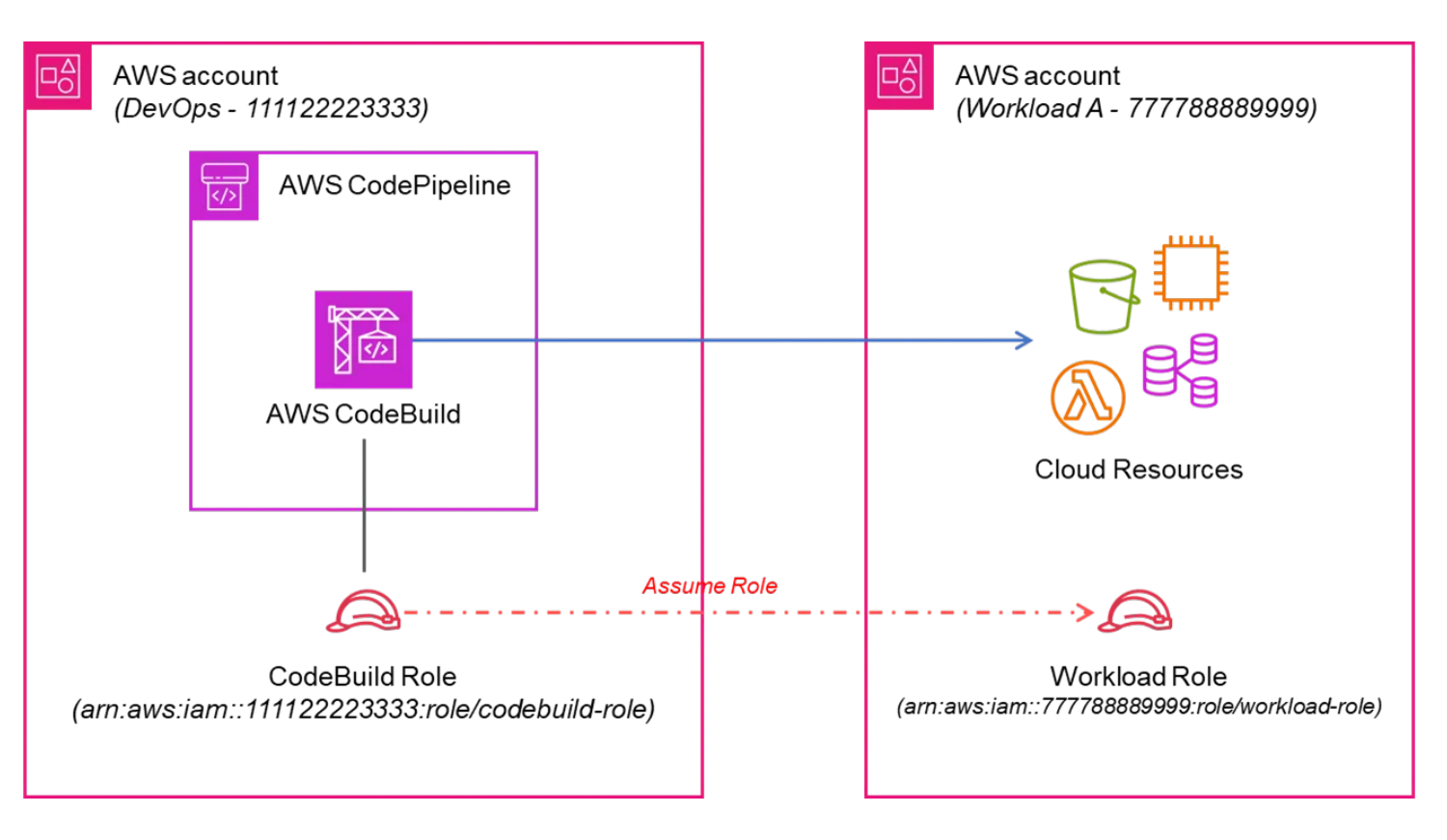
Centralised DevOps to Workload accounts

In our diagram above, we have an AWS CodePipeline used to orchestrate AWS CodeBuild stages to deploy resources into different workload accounts. AWS CodeBuild uses an IAM Role during execution to have permission to perform the necessary actions.

However, the IAM scope will only allow it to deploy into its account by default. Deploying into other AWS accounts would require the CodeBuild IAM Role to assume an IAM role in each workload account.

IAM Permissions

First, let's look at the AWS CodeBuild IAM role in the DevOps account (*111122223333*).



First, the *arn:aws:iam:111122223333:role/codebuild-role* will need permissions to perform the `sts:AssumeRole` action to assume the Workload account. This is how the IAM Role and permissions would look like in Terraform.

```
resource "aws_iam_role" "codebuild_role" {
  name      = "codebuild-role"
  assume_role_policy = jsonencode({
    "Version" : "2012-10-17",
    "Statement" : [
      {
        "Action" : "sts:AssumeRole",
        "Principal" : {
          "Service" : ["codebuild.amazonaws.com"]
        },
        "Effect" : "Allow"
      }
    ]
  })
}

resource "aws_iam_policy" "assume_role_policy" {
  name        = "assume_role_policy_for_777788889999"
  description = "Assume Role permission for Account 777788889999"

  policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Resource": "arn:aws:iam:777788889999:role/workload-role",
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "cross_account_demo"
        }
      }
    }
  ]
}
EOF
}

resource "aws_iam_role_policy_attachment" "attach_permissions" {
  role       = aws_iam_role.codebuild_role.name
  policy_arn = aws_iam_policy.assume_role_policy.arn
}
```

Next, the workload role needs to trust the DevOps CodeBuild role to assume to. Below is what the IAM Role Trust Policy would look like.

```
resource "aws_iam_role" "workload_role" {
  name      = "workload-role"
  assume_role_policy = jsonencode({
    "Version" : "2012-10-17",
    "Statement" : [
      {
        "Action" : "sts:AssumeRole",
        "Effect" : "Allow",
        "Principal": {
          "AWS": [
            "arn:aws:iam::111122223333:role:/codebuild_role",
          ]
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "admin_access" {
  role       = aws_iam_role.lambda_iam_role.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

In our example, we are providing the workload role with administrator access. However, note that this is not recommended if operating in a Production environment.

Cross-Account using Terraform

Once the IAM permissions have been set up, we can now use Terraform to perform cross-account actions.

First, we need to provide a Terraform Provider block. This block provides Terraform with the information needed to perform cross-account roles.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
    }
  }
}

provider "aws" {
  region = "ap-southeast-1"
  alias = "workload_a"
  assume_role {
    role_arn = "arn:aws:iam::777788889999:role/workload-role"
    external_id = "cross_account_demo"
  }
}
```

Once we have the provider block, we can use this Provider when declaring a resource block. Take note that the alias is used to uniquely identify the provider to use.

```
resource "aws_s3_bucket" "s3_bucket" {
  provider = aws.workload_a
  bucket   = "bucket_in_workload_a"
}
```

When we perform a `terraform init` and a `terraform apply` , the resource will be created in the Workload account. Behind the scenes, Terraform will assume a role based on the Provider block to create the requested resource.

If we need to create resources in other accounts, we need to repeat the Provider block and inject the required provider.

```
provider "aws" {
  region = "ap-southeast-1"
  alias = "workload_b"
  assume_role {
    role_arn = "arn:aws:iam::777788889999:role/workload-role"
    external_id = "cross_account_demo"
  }
}

resource "aws_s3_bucket" "s3_bucket" {
  provider = aws.workload_b
  bucket   = "bucket_in_workload_a"
}
```

Conclusion

Having a centralised resource provisioning pipeline helps to facilitate deployments across multiple AWS accounts. Centralisation of the Cloud resource deployment strategy allows organisation to easily enforce standards and best practices in a consistent manner. Terraform as an Infrastructure-as-Code tooling provides an easy way to perform cross-account deployments in a

AWS Cross Account Terraform Iam Sts

24 1

Written by KangZheng Li

23 Followers · 2 Following

Cloud Enthusiast

Subscribe

Responses (1)

Write a response

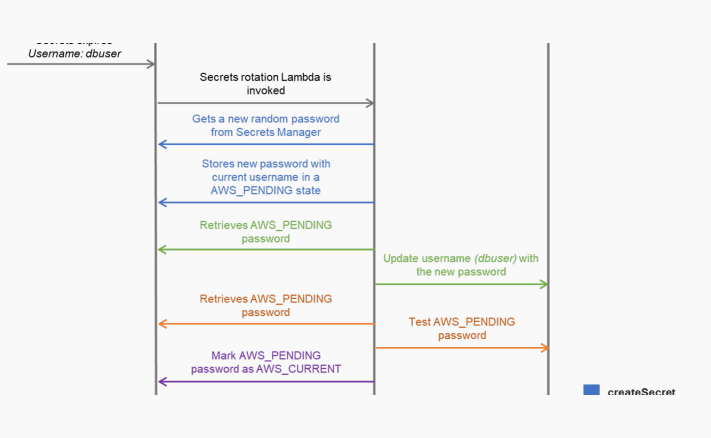
What are your thoughts?

Prakul Singh
May 28, 2024

Just to get it straight - roles can be assumed intrinsically via Terraform, allowing an application or 3rd party to assume the correct roles necessary if a user chooses to allow the application/3rd party to do so? We've been considering the... more

Reply

More from KangZheng Li

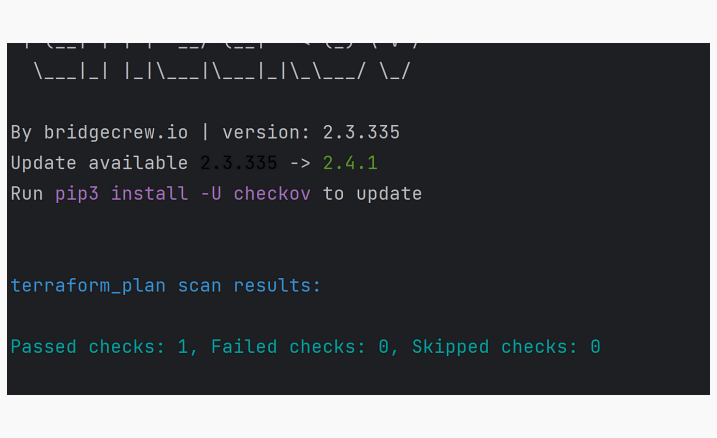


KangZheng Li

AWS Secrets Manager Multi-User Rotation

Introduction

Aug 19, 2023 81

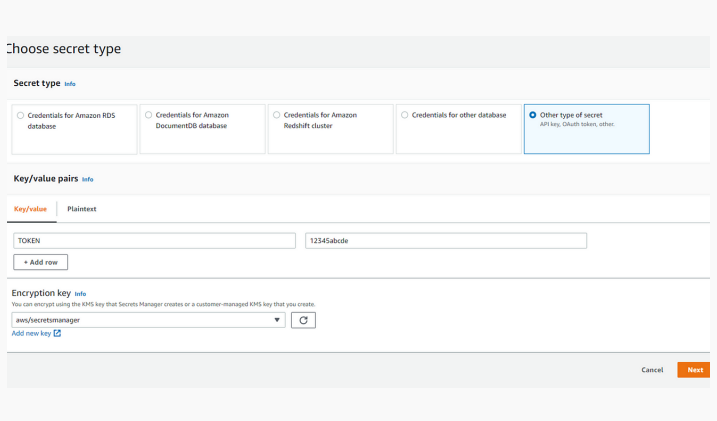


KangZheng Li

Custom Checkov Scans

Introduction

Aug 16, 2023 4 2

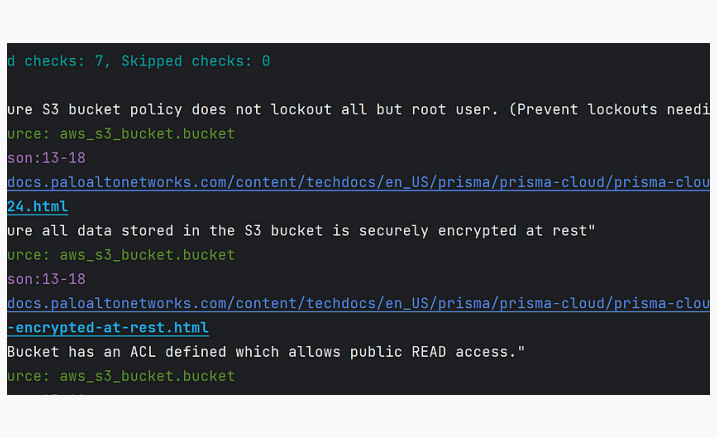


KangZheng Li

Secrets Management for Python on AWS

I came from an application development background and during the early stage of m...

Oct 21, 2022



KangZheng Li

Securing AWS Environments Using Preventive Controls

In this article, I will discuss using Checkov, a Policy-as-Code tool, to help validate AWS...

Jul 23, 2023 7

See all from KangZheng Li

Recommended from Medium

Sandip Das

Common Terraform Mistakes You Should Avoid!

Terraform is an open-source Infrastructure as Code (IaC) tool that allows you to define,...

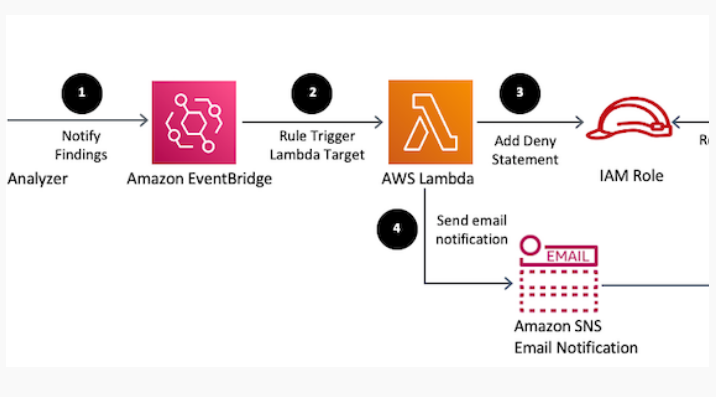
Oct 1, 2024 3

In AWS in Plain English by Alice the Architect

IAM Resource-Based Policies vs. IAM Roles for Cross-Account...

Learn the key differences between IAM resource-based policies and IAM roles for...

Nov 6, 2024 4

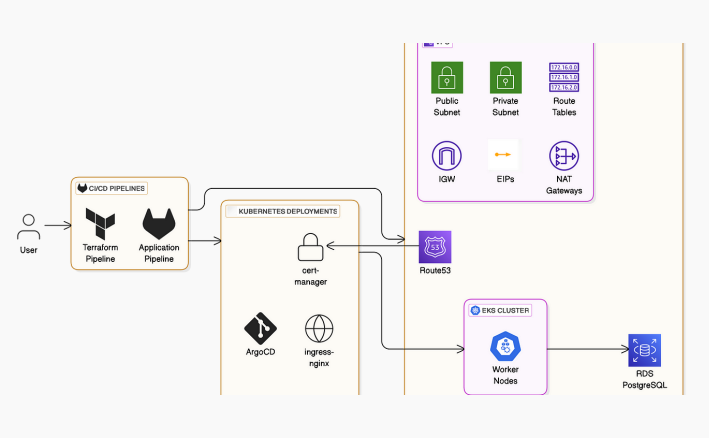


Nagarjun (Arjun) Nagesh

IAM Access Analyzer Remediation

In today's cloud-driven landscape, managing access permissions is paramount to...

Feb 20



In FAUN—Developer Commu... by Mohamed ElE...

Hands-On DevOps Project: Practice Full-Stack Deployment o...

Hands-On DevOps Project: Practice Full-Stack Deployment on AWS EKS with...

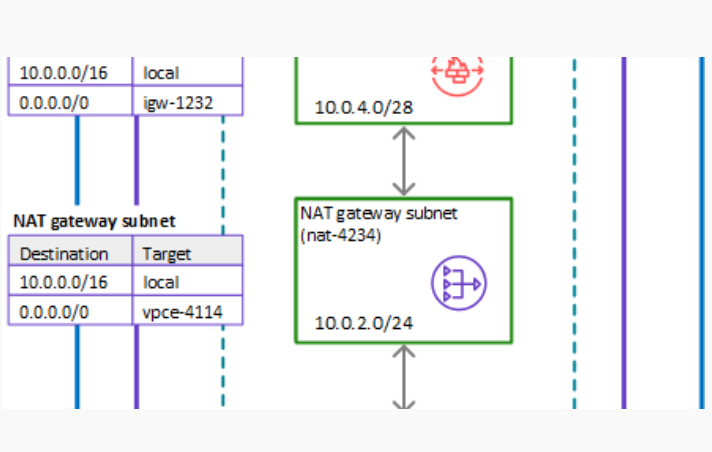
Feb 18 51 1

In AWS in Plain English by Penny Han

Best Practices for Using AWS Secrets Manager

If you happen to need AWS Secrets Manager or are already a user, feel free to check out ...

Oct 6, 2024 74 2



In Towards Dev by Nagarjun (Arjun) Nagesh

Integrating AWS Network Firewall with Internet and NAT Gateways

To enhance security, you can integrate AWS Network Firewall into your VPC architecture...

Mar 20 6

See more recommendations